Ensemble Learning & Support Vector Machines

STAT3612: Statistical Machine Learning

Yinghao Zhu

HKU

November 2026

Outline & Companion Notebook

1. Part 1: Ensemble Learning
   - Design Philosophy & Variance Reduction
   - Boosting: AdaBoost, GBDT & XGBoost
2. Part 2: Support Vector Machines (SVM)
   - Margin Geometry & Kernels
   - Kernel Comparison & Hyperparameters
3. Part 3: Final Group Project Starter
   - End-to-End Baseline Pattern

Hands-On Companion: `ensemble_svm_tutorial.ipynb`

- **Part 1 (Ensemble Learning):** Design philosophy recap, `DecisionTree` depth diagnosis, `Bagging` (`oob_score`), `RandomForest`, `AdaBoost`, `GBDT`, and `XGBoost`.
- **Part 2 (Support Vector Machines):** Margin geometry, `linear`/`poly`/`rbf`/`sigmoid` kernels, (C, γ) sensitivity, and `StandardScaler` pipelines.
- **Part 3 (Group Project Starter):** Multi-table aggregation, `GroupKFold`, and `log_loss` baseline.

Design Recap: Why Combine Trees?

Core Principle

A single unpruned decision tree is a **low-bias, high-variance** estimator: small perturbations in the training data alter early splits and produce drastically different trees.

1. Parallel Ensembles (Bagging / RF)

- **Base Learner:** Deep, low-bias trees trained independently on bootstrap resamples.
- **Aggregation:** Uniform averaging / majority vote $\hat{f}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} h_b(\mathbf{x})$.
- **Objective:** Reduce Variance by cancelling out idiosyncratic noise across trees.

2. Sequential Ensembles (Boosting)

- **Base Learner:** Shallow, high-bias weak learners ($\texttt{max_depth} \in \{1, 3\}$).
- **Aggregation:** Additive updates $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, h_m(\mathbf{x})$ targeting prior errors.
- **Objective:** Reduce Bias iteratively while controlling variance via shrinkage η.

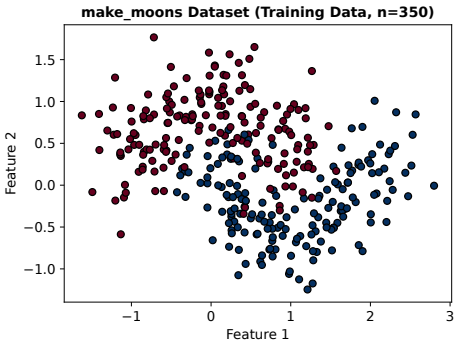
Hands-On Setup: make_moons & Boundary Visualization

1. Generate Synthetic Non-Linear Data:

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.inspection import DecisionBoundaryDisplay
X, y = make_moons(n_samples=500, noise=0.3,
random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.3, random_state=42)
```

2. Reusable 2D Boundary Helper:

```
def plot_decision_boundary(clf, X, y, ax, title):
  DecisionBoundaryDisplay.from_estimator(
    clf, X, response_method="predict",
    cmap=plt.cm.RdBu, alpha=0.5, ax=ax)
  ax.scatter(X[:, 0], X[:, 1], c=y,
    cmap=plt.cm.RdBu, edgecolors="k")
  ax.set_title(title)
```



- 350 train / 150 test samples; noise $= 0.3$ creates class overlap.

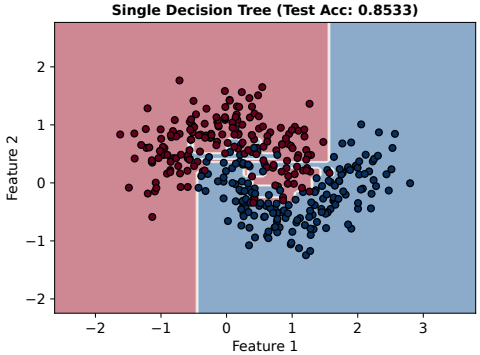
Baseline: Unrestricted Decision Tree (`max_depth=None`)

Core Code (`DecisionTreeClassifier`):

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
# Default max_depth=None grows until leaves are pure
tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)
y_pred_tree = tree_clf.predict(X_test)
tree_accuracy = accuracy_score(y_test, y_pred_tree)
# Train Acc:  1.0000 | Test Acc:  0.8533
```

Diagnostic Observation:

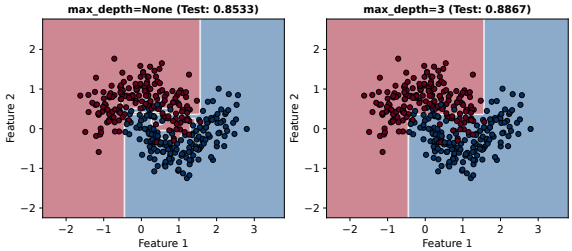
- Axis-aligned splits ($x_j \leq t$) carve narrow, jagged rectangles around isolated noisy points.
- 100% training accuracy vs. **0.8533** test accuracy confirms severe overfitting.



Illustrative Example: Controlling Tree Depth (max_depth)

Pre-Pruning via max_depth=3:

```
# Restrict tree depth to regularize capacity
tree_pruned = DecisionTreeClassifier(
  max_depth=3, random_state=42)
tree_pruned.fit(X_train, y_train)
pruned_acc = accuracy_score(
  y_test, tree_pruned.predict(X_test))
# max_depth=None -> Test Acc:  0.8533
# max_depth=3 -> Test Acc:  0.8867
```



- Capping max_depth eliminates noisy pockets, but a single shallow tree is still restricted to coarse rectangular steps.

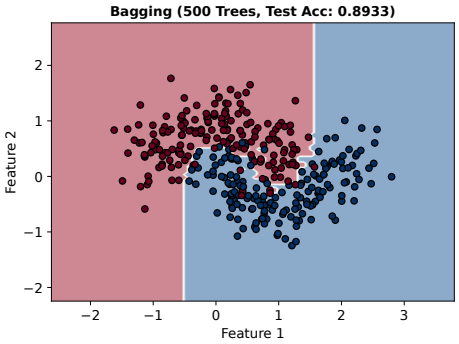
Bagging: Bootstrap Aggregation & Out-of-Bag Evaluation

Core Code (`BaggingClassifier`):

```
from sklearn.ensemble import BaggingClassifier
bag_clf = BaggingClassifier(
  DecisionTreeClassifier(random_state=42),
  n_estimators=500, max_samples=1.0,
  bootstrap=True, oob_score=True,
  n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
# OOB Acc:  bag_clf.oob_score_ -> 0.8971
# Test Acc:  accuracy_score(...)  -> 0.8933
```

Practical Highlights:

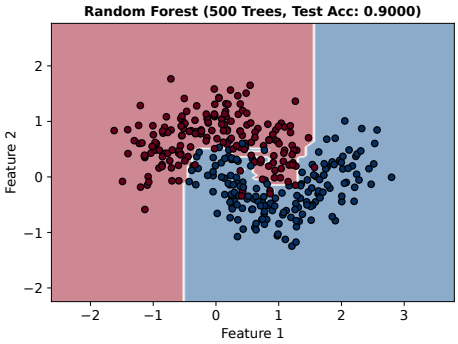
- Averaging 500 deep bootstrap trees smooths the boundary and lifts test accuracy to **0.8933**.
- `oob_score=True`: Each tree leaves out $\approx 36.8\%$ (e^{-1}) of rows, giving a free validation estimate (`oob_score_`).



Random Forest: Decorrelating Splits (`RandomForestClassifier`)

Core Code & Feature Importance:

```
from sklearn.ensemble import RandomForestClassifier
rnd_clf = RandomForestClassifier(
  n_estimators=500, max_features="sqrt",
  oob_score=True, n_jobs=-1, random_state=42)
rnd_clf.fit(X_train, y_train)
rf_acc = accuracy_score(y_test,
rnd_clf.predict(X_test))
# Random Forest Test Acc:  0.9000
# Feature importances:  rnd_clf.feature_importances_
```



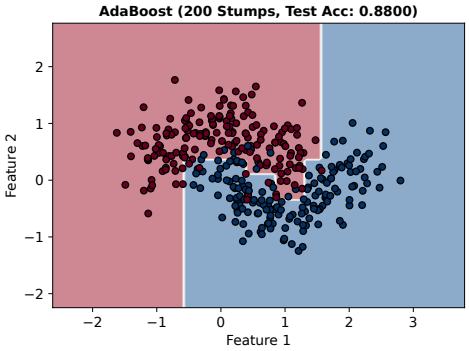
Why It Improves on Plain Bagging:

- Ensemble variance is $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$.
- Sampling a random subset of features (`max_features`) at each split decorrelates trees (lowers ρ), reaching **0.9000**.

Boosting I: Adaptive Re-Weighting (AdaBoostClassifier)

Core Code (200 Decision Stumps):

```
from sklearn.ensemble import AdaBoostClassifier
# Base learner:  depth-1 decision stump
ada_clf = AdaBoostClassifier(
  DecisionTreeClassifier(max_depth=1,
random_state=42),
  n_estimators=200, learning_rate=0.5,
  random_state=42)
ada_clf.fit(X_train, y_train)
ada_acc = accuracy_score(y_test,
ada_clf.predict(X_test))
# AdaBoost Test Acc:  0.8800
```



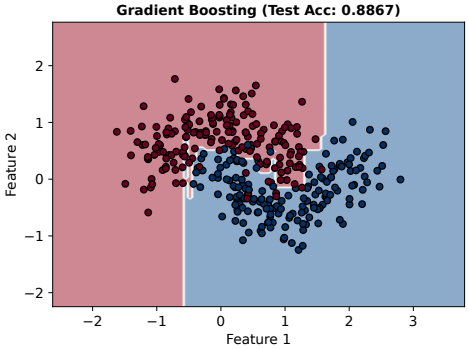
Behavior & Caveat:

- Sequentially fits `max_depth=1` stumps on re-weighted samples, achieving **0.8800**.
- **Caveat:** Exponential loss is sensitive to severe label noise and outliers.

Boosting II: Gradient Boosted Trees (`GradientBoostingClassifier`)

Core Code (Residual Fitting):

```
from sklearn.ensemble import
GradientBoostingClassifier
# Sequential depth-3 trees fit pseudo-residuals
gbrt_clf = GradientBoostingClassifier(
  n_estimators=100, learning_rate=0.1,
  max_depth=3, random_state=42)
gbrt_clf.fit(X_train, y_train)
gbrt_acc = accuracy_score(y_test,
gbrt_clf.predict(X_test))
# Gradient Boosting Test Acc:  0.8867
```



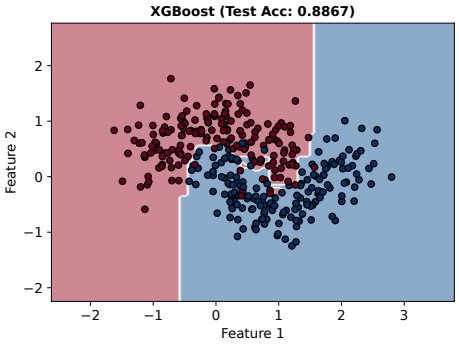
Design Mechanism:

- Instead of re-weighting samples, tree m fits the negative gradient $-\partial\mathcal{L}/\partial F_{m-1}(\mathbf{x}_i)$.
- Shallow trees (`max_depth=3`) capture feature interactions while `learning_rate=0.1` regularizes step size (**0.8867**).

Boosting III: Scalable Regularized GBDT (XGBoost)

Core Code (`xgb.XGBClassifier`):

```
import xgboost as xgb
xgb_clf = xgb.XGBClassifier(
  n_estimators=100, learning_rate=0.1, max_depth=3,
  subsample=1.0, colsample_bytree=1.0,
  eval_metric="logloss", random_state=42)
xgb_clf.fit(X_train, y_train)
xgb_acc = accuracy_score(y_test,
xgb_clf.predict(X_test))
# XGBoost Test Acc:  0.8867
```



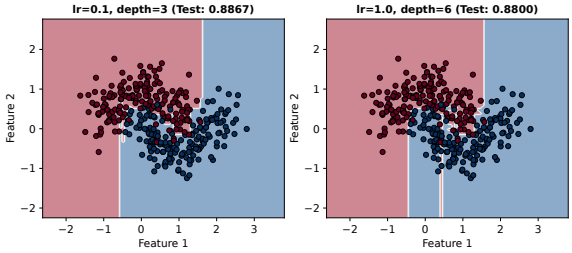
Why `XGBoost` Dominates Tabular Benchmarks:

- Uses 2nd-order Taylor gradients (Hessian) + explicit leaf regularization (`reg_lambda`, `reg_alpha`).
- Built-in row/column subsampling (`subsample`, `colsample_bytree`) and native missing-value routing.

Illustrative Example: Boosting Hyperparameter Sensitivity

Shrinkage vs. Tree Depth Trade-Off:

```
# Good:  small lr + shallow trees
g_good = GradientBoostingClassifier(
  learning_rate=0.1, max_depth=3,
  n_estimators=100, random_state=42)
# Overfit:  large lr + deep trees
g_over = GradientBoostingClassifier(
  learning_rate=1.0, max_depth=6,
  n_estimators=200, random_state=42)
```



- Unlike Random Forest, Boosting can overfit if `learning_rate` or `max_depth` is too large (carving noisy vertical strips, $0.8867 \to 0.8800$).
- **Rule of thumb:** Keep `max_depth` $\in [3, 6]$ and `learning_rate` $\in [0.01, 0.1]$.

Part 1 Summary & Assignment 3 (Q1–Q2) Code Pattern

Benchmark Results on `make_moons`:

Model	Test Acc.
Random Forest (500 Trees)	**0.9000**
Bagging (500 Trees)	0.8933
Pruned Tree (`max_depth` = 3)	0.8867
XGBoost (`max_depth` = 3)	0.8867
Gradient Boosting (`max_depth` = 3)	0.8867
AdaBoost (200 Stumps)	0.8800
Unrestricted Tree (`None`)	0.8533

Assignment 3 (Q1–Q2) Template:

```
from sklearn.tree import plot_tree
# Q1(c-d):  Entropy split + tree visualization
dt = DecisionTreeClassifier(
 max_depth=3, criterion="entropy")
dt.fit(X_train, y_train); plot_tree(dt, filled=True)
# Q2(a-b):  Sweep max_depth = 2..12
for d in range(2, 13):
 t = DecisionTreeClassifier(max_depth=d,
random_state=2025)
 rf = RandomForestClassifier(n_estimators=80,
  max_features=4, max_depth=d, random_state=2025)
```

Design Recap: Maximum Margin & The Kernel Trick

Core Principle

Instead of axis-aligned splits, an SVM finds the decision hyperplane $f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) + b = 0$ that maximizes the geometric margin $\frac{2}{\|\mathbf{w}\|}$ between classes.

- **Sparse Solution via Support Vectors (`clf.support_vectors_`):**
  - Only training points on or inside the margin boundaries $f(\mathbf{x}) = \pm 1$ ($y_i f(\mathbf{x}_i) \leq 1$, circled in plots) have non-zero dual weights $\alpha_i > 0$.
- **Two Key Hyperparameters in `sklearn.svm.SVC`:**
  - **Penalty `C`:** Small `C` prioritizes a wider margin (smooth boundary); large `C` heavily penalizes slack violations ξ_i (harder margin).
  - **Kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$:** Implicitly maps inputs to a higher-dimensional feature space without computing $\phi(\mathbf{x})$ explicitly.

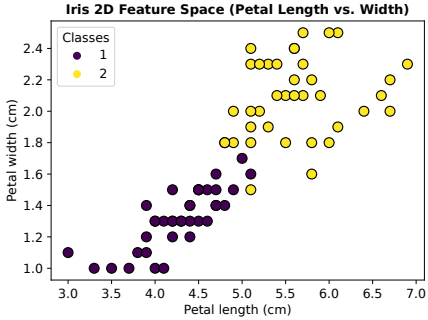
Hands-On Setup: Iris 2D Feature Space & Margin Contours

1. Select 2 Features & 2 Classes from Iris ($N = 100$):

```
data = load_iris()
X = data.data[:, [2, 3]] # petal length & width
y = data.target
idx = np.where((y == 1) | (y == 2)) # versicolor vs
virginica
X, y = X[idx], y[idx]
X_train, X_test, y_train, y_test = train_test_split(
  X, y, random_state=42, test_size=0.25, stratify=y)
```

2. Plot Boundary (0), Margins (± 1) & Support Vectors:

```
clf = svm.SVC(kernel=kernel, gamma=2).fit(X, y)
DecisionBoundaryDisplay.from_estimator(clf, X, ax=ax,
  response_method="decision_function",
  plot_method="contour", levels=[-1, 0, 1],
  colors=["k", "k", "k"], linestyles=["-", "-", "-"])
ax.scatter(clf.support_vectors_[:, 0],
clf.support_vectors_[:, 1],
  s=150, facecolors="none", edgecolors="k")
```



- Solid line: $f(\mathbf{x}) = 0$; dashed lines: $f(\mathbf{x}) = \pm 1$; circles: `support_vectors_`.

Kernel Comparison I: Linear vs. Polynomial Kernel

1. Linear Kernel (Test Acc = 0.88):

$$K(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1^T \mathbf{x}_2$$

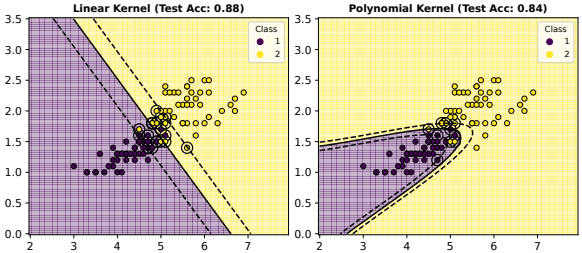
```
mmc_clf = SVC(kernel="linear", random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.88
```

2. Polynomial Kernel (Test Acc = 0.84):

$$K(\mathbf{x}_1, \mathbf{x}_2) = (\gamma \mathbf{x}_1^T \mathbf{x}_2 + r)^d$$

```
mmc_clf = SVC(kernel="poly", random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.84
```

- Default degree = 3 curves the boundary and narrows the margin around the overlap zone.



Kernel Comparison II: RBF vs. Sigmoid Kernel

3. RBF / Gaussian Kernel (Test Acc = 0.92):

$$K(\mathbf{x}_1, \mathbf{x}_2) = \exp\left(-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2\right)$$

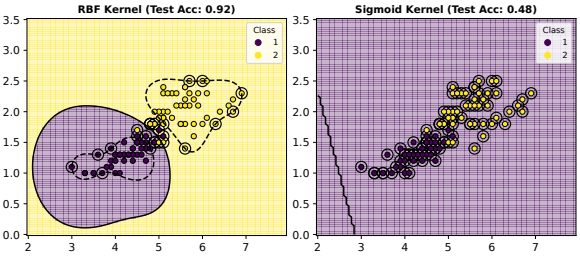
```
mmc_clf = SVC(kernel="rbf", random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.92 (Best!)
```

4. Sigmoid Kernel (Test Acc = 0.48):

$$K(\mathbf{x}_1, \mathbf{x}_2) = \tanh(\gamma \mathbf{x}_1^T \mathbf{x}_2 + r)$$

```
mmc_clf = SVC(kernel="sigmoid",
random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.48
```

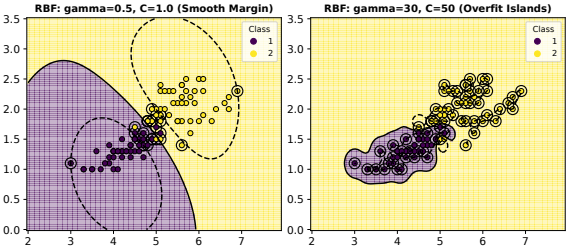
- **RBF** (default in `SVC`) achieves **0.92**; Sigmoid fails here (**0.48**).



Illustrative Example: Tuning C and gamma in RBF SVM

How (C, γ) Shape the Boundary:

```
# Smooth margin:  moderate gamma & C
svc_smooth = SVC(
  kernel="rbf", gamma=0.5, C=1.0).fit(X, y)
# Overfit islands:  large gamma & C
svc_overfit = SVC(
  kernel="rbf", gamma=30, C=50.0).fit(X, y)
```

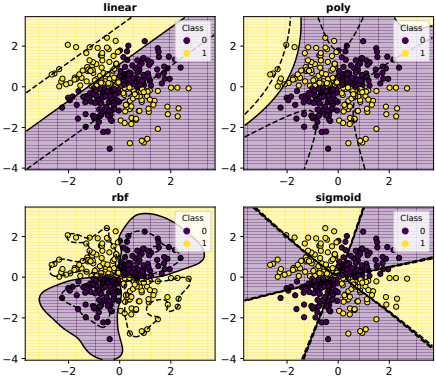


- **Large γ:** Shrinks the Gaussian radius $1/\sqrt{\gamma}$, so each support vector only influences its immediate neighborhood (isolated islands).
- **Large C:** Forces zero training errors at the cost of a highly contorted boundary.

Why Non-Linear Kernels Matter: The 4-Quadrant XOR Benchmark

Core Code (Alternating Quadrants):

```
np.random.seed(0)
X = np.random.randn(300, 2)
# Class 1 when x1 and x2 have opposite signs
y = np.logical_xor(X[:, 0] > 0, X[:, 1] > 0)
_, ax = plt.subplots(2, 2, figsize=(8, 8))
args = dict(long_title=False, support_vectors=False)
for k, a in zip(
  ["linear", "poly", "rbf", "sigmoid"], ax.flat):
  plot_training_data_with_decision_boundary(
    k, a, **args)
```



- **Linear** and **Sigmoid** fail completely on non-linearly separable XOR quadrants.
- **RBF** cleanly separates all four quadrants with closed contours.

Practical SVM Pipeline: Feature Scaling & Assignment 3 (Q4)

Why Scaling is Mandatory for SVM:

- **Trees / Ensembles:** Split rules $x_j \leq t$ only depend on rank ordering $\Longrightarrow$ scale-invariant.
- **SVM:** Margin $\|\mathbf{w}\|^2$ and RBF kernel $\|\mathbf{x}_1 - \mathbf{x}_2\|^2$ are scale-sensitive; unscaled features with large ranges dominate the distance.
- **Best Practice:** Fit `StandardScaler` on `X_train` only, or wrap inside `make_pipeline`.

Assignment 3 (Q4) Template:

```
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix
# Q4(a-b): Scale features & fit RBF SVC
scaler = StandardScaler()
X_tr = scaler.fit_transform(X_train)
X_te = scaler.transform(X_test)
svc = SVC(C=1.5, kernel="rbf").fit(X_tr, y_train)
# Q4(c-d): Support vectors & confusion matrix
n_sv = len(svc.support_vectors_)
cm = confusion_matrix(y_test, svc.predict(X_te))
```

Group Project Starter I: Multi-Table Features & `GroupKFold`

Task Recap: Predicting a Short-Term Learning-Platform Pause

Given Days 0–28 logs (`train.csv`, `vle_events.parquet`, `past_assessments.parquet`), predict probability $P(\texttt{target} = 1)$ of inactivity on Days 29–42, evaluated by **Binary Log Loss**.

```
# 1. Aggregate event logs to sample_id level
vle_agg = vle.groupby("sample_id").agg(
  total_clicks=("clicks", "sum"),
  active_days=("day", "nunique")).reset_index()
df = train.merge(vle_agg, on="sample_id", how="left")
# 2. GroupKFold prevents student_group leakage!
gkf = GroupKFold(n_splits=5)
for tr, va in gkf.split(
  X, y, groups=df["student_group"]):
  clf.fit(X.iloc[tr], y.iloc[tr])
  p_val = clf.predict_proba(X.iloc[va])[:, 1]
  score = log_loss(y.iloc[va], p_val)
```

Two Critical Pitfalls to Avoid:

- **Leakage:** Repeated students share `student_group`; always split by group rather than random `train_test_split`.
- **Metric:** Log loss requires calibrated probabilities (`predict_proba[:, 1]`), never hard 0/1 labels (`predict`).

Group Project Starter II: Applying Today's Models & Next Steps

Plugging In RF, XGBoost & SVM Baselines:

```
# 1. Tree ensembles handle mixed tabular scales
rf = RandomForestClassifier(
  n_estimators=300, max_depth=8, min_samples_leaf=10)
xgb_m = xgb.XGBClassifier(
  n_estimators=300, learning_rate=0.05, max_depth=4)
# 2. SVM needs imputation + scaling + Platt
svm_pipe = make_pipeline(SimpleImputer(),
  StandardScaler(), SVC(C=1.0, probability=True))
# 3. Export clipped probabilities to submission.csv
sub["prediction"] = np.clip(
  xgb_m.predict_proba(X_te)[:, 1], 1e-6, 1 - 1e-6)
```

What to Explore Next (Your Turn!):

- **Temporal Dynamics:** Compare Week 4 clicks (Days 22–28) vs. Weeks 1–3 to capture recent drop-off trends.
- **Assessment Behavior:** Late submission rate (`submitted_day` > `due_day`) and weighted completion.
- **Calibration & Ensembling:** Blend `XGBoost` + `RandomForest` probabilities to reduce log loss.

Take-Home Summary: Ensembles & SVM Cheat Sheet

Method	Mechanism	Primary Effect	Key `sklearn` Args	Scale?
Decision Tree	Recursive axis splits	Low bias, high var.	`max_depth`, `criterion`	No
Bagging	Parallel bootstrap trees	Reduces variance	`n_estimators`, `oob_score=True`	No
Random Forest	Bagging + random features	Decorrelates trees	`n_estimators`, `max_features`	No
AdaBoost	Sequential re-weighted stumps	Reduces bias	`n_estimators`, `learning_rate`	No
GBDT / XGBoost	Sequential trees on residuals	Reduces bias	`max_depth=3`, `learning_rate`	No
Linear SVM	Max-margin hyperplane	Global linear margin	`kernel="linear"`, `C`	**Yes**
Kernel SVM (RBF)	Margin in $\phi(\mathbf{x})$ space	Smooth non-linear	`kernel="rbf"`, `C`, `gamma`	**Yes**

Next Steps

Run `ensemble_svm_tutorial.ipynb` locally for hands-on experiments, **Assignment 3** (due Nov 26), and your **Final Group Project** starter pipeline.